lubridate cheat sheet

MC 451 / MC 501 / MC 427 · turn numbers and text into real dates

lubridate turns raw numbers or text into real date-times you can pull the hour or day out of, round off, or measure between. Each entry says what the function does, shows it on the course Twitch data, and shows the exact output. **On your data** rewrites the call with a plain column name, and **If it breaks** names the likely error.

Every worksheet starts with these lines, so the examples assume them:

```
library(tidyverse)   # lubridate comes with the tidyverse
library(v2v)          # the course package and its Twitch data
options(scipen = 10)  # long numbers print in full, so timestamps are readable
```

The data these examples use

In the Twitch data, `date` is stored as a count of **milliseconds** since January 1, 1970, a thirteen-digit number. That is why the first step always divides by 1000.

```
chat %>%
  select(channel, date) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 2
  channel             date
  <chr>              <dbl>
1 sodapoppin 1542578127023
2 xqcow      1542578135562
3 forsen     1542578143610
4 sodapoppin 1542578157677
5 xqcow      1542578164789
```

`as_datetime()`: turn a number into a date-time

Reads a count of **seconds** since 1970 and returns a real date-time. The Twitch `date` is in milliseconds, so divide by 1000 first.

```
chat %>%
  mutate(ts = as_datetime(date / 1000)) %>%
```

```
  select(date, ts) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 2
           date ts
          <dbl> <dttm>
1 1542578127023 2018-11-18 21:55:27
2 1542578135562 2018-11-18 21:55:35
3 1542578143610 2018-11-18 21:55:43
4 1542578157677 2018-11-18 21:55:57
5 1542578164789 2018-11-18 21:56:04
```

On your data: `mutate(ts = as_datetime(seconds_column))`

If it breaks: a date near the year 50000 means you forgot to divide milliseconds by 1000. A date near 1970 means you divided a seconds column that did not need it. Print `range(ts)`: every message here falls in November 2018.

`ymd_hms()`: read a date-and-time written as text

Reads text written as year, month, day, hour, minute, second, in that order.

```
ymd_hms("2018-11-18 22:11:53")
```

```
[1] "2018-11-18 22:11:53 UTC"
```

On your data: `ymd_hms("2024-03-01 09:30:00")`

If it breaks: `All formats failed to parse` means the parts are in a different order. Match the function to the order: `mdy_hms()` for month-first text, `dmy_hms()` for day-first.

`ymd()`: read a date with no time

Reads text written as year, then month, then day.

```
ymd("2018-11-18")
```

```
[1] "2018-11-18"
```

On your data: `ymd("2024-03-01")`

If it breaks: `All formats failed to parse` on `03/01/2024` means the parts are month-first; use `mdy()` instead.

`mdy()`: read a month-first date

Reads text written month, day, year, including the long form with a month name.

```
mdy("November 18, 2018")
```

```
[1] "2018-11-18"
```

On your data: `mdy("March 1, 2024")`

If it breaks: a two-digit year like `3/1/24` is read as 2024, which is usually what you want, but check the result when the century matters.

`hour()`: pull the hour out of a date-time

Returns the hour, 0 to 23, from a date-time. `day()`, `minute()`, `wday()`, and `month()` work the same way for the other parts.

```
chat %>%
  mutate(ts = as_datetime(date / 1000), hour = hour(ts)) %>%
  count(hour)
```

```
# A tibble: 24 × 2
    hour     n
   <int> <int>
 1     0  1816
 2     1  1959
 3     2  1013
 4     3   833
 5     4  1384
 6     5  1317
 7     6  1393
 8     7  1165
 9     8  1016
10     9  1381
# ℹ 14 more rows
```

On your data: `mutate(hour = hour(ts))`

If it breaks: `hour()` on a plain number, not a date-time, returns `NA`. Run `as_datetime()` first.

`day()`: pull the day of the month

Returns the day of the month, 1 to 31. This corpus runs from the 18th to the 24th.

```
chat %>%
  mutate(ts = as_datetime(date / 1000), day = day(ts)) %>%
  count(day)
```

```
# A tibble: 7 × 2
    day     n
  <int> <int>
1    18   821
2    19  4826
3    20  7169
4    21  9092
5    22  3561
6    23  8335
7    24  1463
```

On your data: `mutate(day = day(ts))`

If it breaks: do not confuse `day()` (day of the month, a number) with `wday()` (day of the week). Use `wday(ts, label = TRUE)` for `Mon`, `Tue`, and so on.

`floor_date()`: round a time down to a unit

Rounds a date-time down to the start of the unit you name, so every message in the same hour gets the same stamp. Use it to count events per hour or per day.

```
chat %>%
  mutate(ts = as_datetime(date / 1000),
         hour_bin = floor_date(ts, "hour")) %>%
  count(hour_bin) %>%
  slice_head(n = 5)
```

```
# A tibble: 5 × 2
  hour_bin                n
  <dttm>              <int>
1 2018-11-18 21:00:00     9
2 2018-11-18 22:00:00   432
3 2018-11-18 23:00:00   380
4 2018-11-19 00:00:00  1320
5 2018-11-19 01:00:00  1426
```

On your data: `mutate(day_bin = floor_date(ts, "day"))`

If it breaks: an unknown unit name errors. Use `"hour"`, `"day"`, `"week"`, or `"month"`, in quotes.

`interval()`: measure the span between two times

Makes a span between two date-times. Divide it by a unit to get the length as a number. `dminutes(1)` is one minute, `dhours(1)` one hour, `ddays(1)` one day.

```
start <- ymd_hms("2018-11-18 22:00:00")
end   <- ymd_hms("2018-11-18 23:30:00")
interval(start, end) / dminutes(1)
```

```
[1] 90
```

On your data: `interval(start_col, end_col) / ddays(1)`

If it breaks: a negative number means the two times are in the wrong order; the earlier time goes first.

`now()`: the current date and time

Returns the moment you run it, so the value below is simply when this sheet was built; yours will differ.

```
now()
```

```
[1] "2026-11-05 12:39:30 CST"
```

On your data: `now()` takes no arguments. Use `today()` for the date with no time.

If it breaks: nothing to break. Because it changes every run, never paste a `now()` result into a worksheet and expect it to match again.